Machine Learning for Mechanical Engineers at CoEP Session 19: ME Apps

Session 19: ME Apps

Applications to Mechanical Engineering

The Announcement

"Revitalizing manufacturing through AI"

- AI is already transforming the IT industry.
- It is now time to for an AI-powered society.
- We will initially focus on the manufacturing industry.

(Reference: "Medium" - Andrew Ng, Dec 14,2017)

Why Manufacturing?

- Manufacturing touches every part of society.
- Need to help not just bits and pixels but physical world.

Challenges facing manufacturing

- Variable quality and yield,
- Inflexible production line design
- Inability to manage capacity
- and rising production costs.

ML to help address them.

Challenges facing manufacturing

More help in

- Shorten design cycles,
- Remove supply-chain bottlenecks,
- Reduce materials and energy waste,
- and improve production yields.

Why now?

"Industry today is experiencing a never seen increase in available data"

Sources:

- Environmental data
- Sensor data from production line
- Machine tool parameters
- Logistics, transaction data
- Customer demand data.

(Reference: What is smart manufacturing? Time Magazine - Chand & Davis, 2010)

How ML can help?

- Ability to handle high-dimensional problems
- SVM handling high dimensionality (> 1000) very well.
- Concerns: Over-fitting

(Ref: "Machine learning in manufacturing: advantages, challenges, and applications"- Thorsten Wuest, et al)

How ML can help?

- Ability to reduce complexity
- Present transparent and concrete advice for practitioners
- E.g. monitor XX and parameter YY at checkpoint ZZ
- ML finds important features, thresholds

(Ref: "Machine learning in manufacturing: advantages, challenges, and applications"- Thorsten Wuest, et al)

How ML can help?

- Ability to adapt to changing environment
- New data, new training, new model.

(Ref: "Machine learning in manufacturing: advantages, challenges, and applications"- Thorsten Wuest, et al)

New wave: IoT

"Internet of Things"

- IOT is connecting things to the Internet
- Not the same as connecting things to the cellular network!
- IOT personal (Health apps)
- IOT Machine (Machine 2 Machine M2M)
- IoT Everywhere (5G, wifi)

New wave: IoT

IoT Data

- Sensors everywhere.
- By 2020,50 billion connected devices.
- Wearables, cars, machines, cities.

New wave: IoT

IoT and Machine Learning

- Basic idea of machine learning is to build a mathematical model based on training data(learning stage) , predict results for new data(prediction stage) and tweak the model based on new conditions
- IoT creates a lot of contextual data
- Use the scale of IoT data to gain new insights

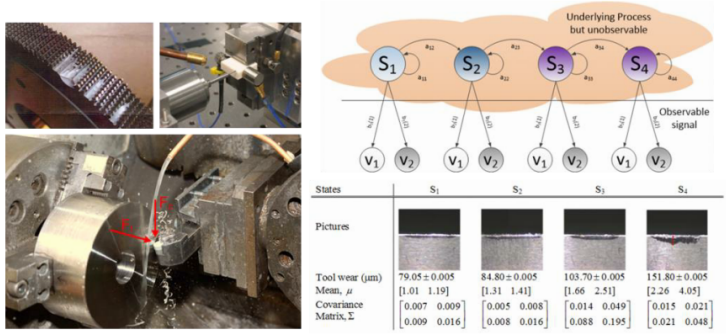
New wave: IoT

IoT ML Applications

- Consumer : bio sensors(real time tracking)
- Enterprise : Complex machinery (preventative maintenance), asset efficiency.
- Public infrastructure: Smart Cities, Dynamically adjust traffic lights

Sample Applications: Tool Wear Estimation

Hidden Markov Model



S. Lee, L. Li*, and J. Ni, 2010, "Online Degradation Assessment and Adaptive Fault Detection Using Modified Hidden Markov Model," ASME Journal of Manufacturing Science and Engineering, 132(2), pp. 021010-11. [PDF]

Ideas for new applications

Project ideas?

- Kaggle: https://www.kaggle.com/datasets
- UCI: http://archive.ics.uci.edu/ml/index.php
- Other: http://deeplearning.net/datasets/

Linear Regression — House Price Prediction

(Ref: scikit-learn User Guide — Generalized Linear Models)

Problem Info

- Dataset: California Housing (`sklearn.datasets.fetch_california_housing`, built-in)
- Features: median income, house age, avg rooms, avg bedrooms, population, latitude, longitude
- Target: Median house value (in 100,000s USD)
- Goal: Predict house prices using Linear Regression; compare Ridge and Lasso regularization

Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
```

Load and Explore Data

```
data = fetch_california_housing()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['MedHouseVal'] = data.target

print(df.shape)       # (20640, 9)
print(df.describe())
print(df.head())
```

Train Linear Regression

```
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred = lr.predict(X_test)

print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2  :", r2_score(y_test, y_pred))
```

Compare Regularization

```
for name, model in [("Linear", LinearRegression()),
                    ("Ridge",  Ridge(alpha=1.0)),
                    ("Lasso",  Lasso(alpha=0.1))]:
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    rmse = np.sqrt(mean_squared_error(y_test, pred))
    r2   = r2_score(y_test, pred)
    print(f"{name:8s}  RMSE={rmse:.4f}  R2={r2:.4f}")
```

Predicted vs Actual

Points close to the diagonal indicate accurate predictions.

```
plt.figure(figsize=(6, 5))
plt.scatter(y_test, y_pred, alpha=0.3, s=10)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()], 'r--', lw=2)
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Linear Regression: Predicted vs Actual")
plt.tight_layout()
plt.show()
```

SVM — Handwritten Digit Recognition

(Ref: scikit-learn SVM example — Digit Classification)

Problem Info

- Dataset: Digits (`sklearn.datasets.load_digits`, built-in)
- 1,797 images of handwritten digits (0–9), each 8×8 pixels = 64 features
- Goal: Classify each image into one of 10 digit classes using SVM
- Demonstrates the power of SVM's RBF kernel for non-linear image classification

Import Libraries

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
```

Load and Visualize Data

```
digits = load_digits()
print("Data shape:", digits.data.shape)   # (1797, 64)
print("Classes   :", digits.target_names) # [0 1 2 ... 9]

fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for ax, img, label in zip(axes.ravel(),
                          digits.images, digits.target):
    ax.imshow(img, cmap='gray_r')
    ax.set_title(str(label))
    ax.axis('off')
plt.suptitle("Sample Digits")
plt.show()
```

Train SVM Classifier

```
X_train, X_test, y_train, y_test = train_test_split(
    digits.data, digits.target,
    test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

svm = SVC(kernel='rbf', C=10, gamma=0.001)
svm.fit(X_train, y_train)
y_pred = svm.predict(X_test)

print(classification_report(y_test, y_pred))
```

Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=digits.target_names,
            yticklabels=digits.target_names)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("SVM Digit Recognition: Confusion Matrix")
plt.tight_layout()
plt.show()
```

Visualize Misclassifications

```
wrong = np.where(y_pred != y_test)[0]
fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for ax, idx in zip(axes.ravel(), wrong[:10]):
    ax.imshow(X_test[idx].reshape(8, 8), cmap='gray_r')
    ax.set_title(f"True:{y_test[idx]} Pred:{y_pred[idx]}")
    ax.axis('off')
plt.suptitle("Misclassified Digits")
plt.tight_layout()
plt.show()
```

K-Means Clustering — Customer Segmentation

(Ref: scikit-learn Clustering User Guide)

Problem Info

- Scenario: A retail company wants to segment its customers for targeted marketing
- Features: Annual Income (k USD) and Spending Score (1–100)
- Data: Synthetic dataset mimicking a retail loyalty programme

- Goal: Discover natural customer groups using K-Means; choose optimal K with the Elbow method

Import Libraries and Generate Data

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

np.random.seed(42)
# Simulate 3 customer segments: budget, standard, premium
centres = [(30, 20), (55, 55), (80, 85)]
X = np.vstack([
    np.random.randn(120, 2) * [8, 10] + c
    for c in centres])
df = pd.DataFrame(X, columns=['AnnualIncome', 'SpendingScore'])
print(df.describe())
```

Elbow Method: Choosing K

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

inertia = []
K_range = range(1, 10)
for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertia.append(km.inertia_)

plt.plot(K_range, inertia, marker='o')
plt.xlabel("Number of Clusters K")
plt.ylabel("Inertia (WCSS)")
plt.title("Elbow Method")
plt.grid(True); plt.show()
```

Fit K-Means with Optimal K

```
km = KMeans(n_clusters=3, random_state=42, n_init=10)
df['Segment'] = km.fit_predict(X_scaled)

labels = {0: 'Budget', 1: 'Standard', 2: 'Premium'}
colours = ['#e07b54', '#5b9bd5', '#70ad47']

plt.figure(figsize=(7, 5))
for seg, (name, colour) in zip([0, 1, 2],
                               zip(labels.values(), colours)):
    mask = df['Segment'] == seg
    plt.scatter(df.loc[mask, 'AnnualIncome'],
                df.loc[mask, 'SpendingScore'],
                label=name, color=colour, alpha=0.7)
plt.xlabel("Annual Income (k USD)")
plt.ylabel("Spending Score")
plt.title("Customer Segments (K-Means, K=3)")
plt.legend(); plt.tight_layout(); plt.show()
```

Segment Profiles

- **Budget**: low income, low spending — price-sensitive customers
- **Standard**: mid income, mid spending — core customer base
- **Premium**: high income, high spending — loyalty programme targets

```
profile = df.groupby('Segment')[
    ['AnnualIncome','SpendingScore']].mean()
profile.index = [labels[i] for i in profile.index]
print(profile.round(1))
```

PCA + K-Means — Handwritten Digit Clustering

(Ref: scikit-learn Decomposition and Clustering User Guide)

Problem Info

- Dataset: Digits (`sklearn.datasets.load_digits`, built-in) — 1,797 images, 64 features each
- Part A: Use PCA to reduce 64 dimensions to 2; visualise digit clusters in 2D
- Part B: Apply K-Means (K=10) on PCA-reduced data; measure cluster quality
- Part C: Vary the number of PCA components (2, 10, 30) and observe effect on clustering

Import Libraries

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, adjusted_rand_score
```

Part A: PCA Visualisation

```
digits = load_digits()
X = StandardScaler().fit_transform(digits.data)

pca2 = PCA(n_components=2)
X2   = pca2.fit_transform(X)

plt.figure(figsize=(8, 6))
```

```
scatter = plt.scatter(X2[:, 0], X2[:, 1],
                      c=digits.target, cmap='tab10',
                      s=10, alpha=0.7)
plt.colorbar(scatter, label='Digit')
plt.title("Digits projected to 2 PCA components")
plt.xlabel("PC 1"); plt.ylabel("PC 2")
plt.tight_layout(); plt.show()

print("Variance explained:", pca2.explained_variance_ratio_.sum())
```

Part B: K-Means on PCA Data

- **Silhouette score**: measures cluster compactness (higher is better, max 1.0)
- **Adjusted Rand Index**: compares clusters to true digit labels (1.0 = perfect)

```
for n_comp in [2, 10, 30]:
    Xr = PCA(n_components=n_comp).fit_transform(X)
    km = KMeans(n_clusters=10, random_state=42, n_init=10)
    labels = km.fit_predict(Xr)
    sil = silhouette_score(Xr, labels)
    ari = adjusted_rand_score(digits.target, labels)
    print(f"PCA={n_comp:2d}  Silhouette={sil:.3f}  "
          f"ARI={ari:.3f}")
```

Assignment Tasks

- Report the variance explained by 2, 10, and 30 PCA components
- Plot the explained variance ratio (scree plot) and identify the "elbow"
- At which number of PCA components does ARI peak? Explain why
- Visualise cluster centres (inverse-transform to pixel space) for the best configuration
- What do the cluster centres look like? Do they resemble recognisable digits?

Assignments for Mechanical Engineering

Predictive Maintenance using Regression Analysis

- **Dataset**: NASA Turbofan Engine Degradation Simulation Data Set
- **Source**: NASA Prognostics Data Repository
- **Description**: Predict the remaining useful life of turbofan engines using regression analysis based on sensor data.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

data = pd.read_csv('turbofan_data.csv')
X = data[['feature1', 'feature2', 'feature3']]
y = data['remaining_useful_life']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

Structural Health Monitoring using Anomaly Detection

- **Dataset**: Bridge Health Monitoring Dataset
- **Source**: UCI Machine Learning Repository
- **Description**: Detect anomalies indicating potential structural failures in bridges based on sensor readings.

```
import pandas as pd
from sklearn.ensemble import IsolationForest

data = pd.read_csv('bridge_health_data.csv')
model = IsolationForest(contamination=0.1)
model.fit(data)
anomalies = model.predict(data)
```

Design Optimization using Genetic Algorithms

- **Dataset**: Engineering Design Optimization Dataset
- **Source**: Kaggle
- **Description**: Optimize engineering designs using genetic algorithms to evaluate performance metrics.

```
from deap import base, creator, tools, algorithms

creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
creator.create("Individual", list, fitness=creator.FitnessMin)

toolbox = base.Toolbox()
# Define operations for crossover, mutation, etc.
population = toolbox.population(n=50)
algorithms.eaSimple(population, toolbox, cxpb=0.5, mutpb=0.2, ngen=40)
```

Fault Detection in Rotating Machinery using SVM

- **Dataset**: Machinery Fault Diagnosis Dataset
- **Source**: UCI Machine Learning Repository
- **Description**: Classify faults in rotating machinery using support vector machines (SVM).

```
import pandas as pd
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split

data = pd.read_csv('machinery_fault_data.csv')
X = data.drop('fault_type', axis=1)
y = data['fault_type']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25)
model = SVC()
model.fit(X_train, y_train)
accuracy = model.score(X_test, y_test)
```

Thermal Analysis using Neural Networks

- **Dataset**: Thermal Conductivity Dataset
- **Source**: Kaggle
- **Description**: Predict thermal conductivity of materials using neural networks.

```
from keras.models import Sequential
from keras.layers import Dense

model = Sequential()
model.add(Dense(64, activation='relu', input_dim=10))
model.add(Dense(32, activation='relu'))
model.add(Dense(1))

model.compile(optimizer='adam', loss='mean_squared_error')
```

Quality Control in Manufacturing using CNNs

- **Dataset**: Manufacturing Defect Dataset
- **Source**: Kaggle
- **Description**: Train a CNN to classify manufacturing defects based on part images.

```
from keras.models import Sequential
from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

model = Sequential()
model.add(Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)))
model.add(MaxPooling2D(pool_size=(2,2)))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))

model.compile(optimizer='adam', loss='categorical_crossentropy',
    metrics=['accuracy'])
```

Energy Consumption Prediction using Time Series Analysis

- **Dataset**: Energy Consumption Dataset
- **Source**: UCI Machine Learning Repository
- **Description**: Forecast energy consumption patterns using ARIMA models.

```
import pandas as pd
from statsmodels.tsa.arima.model import ARIMA

data = pd.read_csv('energy_consumption.csv')
model = ARIMA(data['consumption'], order=(5,1,0))
model_fit = model.fit()
forecast = model_fit.forecast(steps=10)
```

Material Property Prediction using Gaussian Processes

- **Dataset**: Materials Project Database
- **Source**: Materials Project
- **Description**: Predict material properties using Gaussian process regression.

```
from sklearn.gaussian_process import GaussianProcessRegressor

gp = GaussianProcessRegressor()
gp.fit(X_train, y_train)
predictions = gp.predict(X_test)
```

Load Prediction in Structural Engineering using Random Forests

- **Dataset**: Structural Load Data
- **Source**: UCI Machine Learning Repository
- **Description**: Predict structural loads using random forests.

```
import pandas as pd
from sklearn.ensemble import RandomForestRegressor

data = pd.read_csv('structural_load_data.csv')
X = data.drop('load', axis=1)
y = data['load']

model = RandomForestRegressor()
```

```
model.fit(X, y)
predictions = model.predict(X)
```

Simulation of Fluid Dynamics using Deep Learning

- **Dataset**: Fluid Dynamics Simulation Data
- **Source**: OpenFOAM
- **Description**: Simulate fluid dynamics using deep learning models.

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32, (3,3), activation='relu', input_shape=(64,64,1)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(1)
])

model.compile(optimizer='adam', loss='mean_squared_error')
```